



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Parser

CMSC 124 Lab 2

Prepared by: Rene Andre Bedonia Jocsing

Published: September 18, 2026

Deadline: December 09, 2026

Your scanner turns text into tokens. That's the words. This activity is the grammar. The **parser** takes the token stream and works out how those words fit together. If the scanner answers "what are the words?", the parser answers "what do they say?"

The output isn't a list. It's a tree. Building it forces you to be precise about your language in a way the scanner never did. This activity follows Chapters 5 and 6 of Nystrom's *Crafting Interpreters*, "Representing Code" and "Parsing Expressions."

Background

Consider a mathematical expression:

```
1 2 + 3 * 4
```

Your scanner breaks this into `NUMBER(2)`, `PLUS`, `NUMBER(3)`, `STAR`, `NUMBER(4)`. Now what? A flat sequence loses the one fact that matters. Multiplication binds tighter than addition. Read the tokens left to right and nothing stops you from adding 2 and 3 first, which is a mistake you almost certainly made in a math class once. The parser's job is to organize those tokens into a structure where the right answer is the only reading available:

```
1   +
2  / \
3 2   *
4   / \
5 3   4
```

That structure is an **abstract syntax tree**, or AST. It's abstract because it throws away everything about the source text that no longer matters, such as the parentheses you typed and the whitespace between tokens, and keeps only the structure. Evaluation, in Lab 3, is a walk over this tree from the bottom-left leaves up to the root, so `3 * 4` happens first because it's deeper.

Context-Free Grammars

Before you can build a parser you have to write down the rules it enforces. The tool for that is a **context-free grammar**, or CFG. The name is intimidating, but the idea isn't. It's a systematic way to describe which token sequences your language allows.

A CFG has three parts. **Terminals** are the actual tokens your scanner emits, written in quotes. They're called terminal because they end the process. Once you've reached one, there's nothing left to expand. **Non-terminals** are the abstract categories, such as "expression" or "term", written without quotes. They stand for something not yet spelled out. **Production rules** say how a non-terminal can be replaced by a sequence of terminals and other non-terminals.

That leaves the word "context-free," which sounds like the hard part, but is just a promise made to you. It means a non-terminal expands the same way no matter what surrounds it. An `expression` inside a function call, an `expression` on the right of an assignment, and an `expression` sitting alone on a line all follow the identical rule, because the rule never asks what its neighbors are. It doesn't ask for *context*. It's *context-free*. Without that promise, you couldn't write one function per rule and expect it to work everywhere the rule appears, which is the entire technique you're about to learn. Natural languages aren't context-free, roughly why parsing English is a research field and parsing your language is a month.

There's one more idea to have straight before the rules stop looking like decoration. It's the one nobody tells you. A grammar and a parser run the same machine in opposite directions.

Read a grammar forwards and it **generates**. Start with a non-terminal, replace it with one of its productions, keep replacing non-terminals until only terminals remain, and whatever you're left holding is a valid program. Every sequence the grammar can produce this way is legal. Every sequence it can't produce is not. That's what a grammar *is*: a machine for manufacturing valid programs.

A parser runs that machine backwards. It's handed a finished token sequence and has to work out which replacements would have produced it, or report that no series of replacements could have. Recognizing is the harder direction. The rest of this manual is about it. But when you're stuck on why your parser rejects something, the fastest debugging move is to switch directions. Try to generate the input from your grammar by hand, and the step where you get stuck is the rule that's wrong.

Here's a grammar for arithmetic and comparison, taken from the textbook:

```

1  expression → equality
2  equality   → comparison ( ( "!=" | "==" ) comparison )*
3  comparison → term ( ( ">" | ">=" | "<" | "<=" ) term )*
4  term       → factor ( ( "-" | "+" ) factor )*
5  factor     → unary ( ( "/" | "*" ) unary )*
6  unary      → ( "!" | "-" ) unary | primary
7  primary    → NUMBER | STRING | "true" | "false" | "nil" | "(" expression ")"

```

Each line is a production rule. The symbol before the arrow is a non-terminal. What follows the arrow (read it as "produces") is what it can be replaced with. A pipe | means "or". Pick whichever branch fits. The * means zero or more repetitions, + means one or more, and ? means optional. Parentheses group choices. A parenthesis in quotes means the literal parenthesis character.

Those last four are borrowed from regular expressions. They're pure convenience. A grammar in its stripped-down form has only replacement and choice, so repetition has to be spelled with a recursive helper rule:

```

1  term       → factor termTail
2  termTail   → ( "-" | "+" ) factor termTail
3             | nothing

```

Three lines, one extra non-terminal, and a reader who has to hold `termTail` in their head to see that it means "and then some more factors." With the * it collapses back to one line that says the same thing:

```

1  term → factor ( ( "-" | "+" ) factor )*

```

Do the former for every repetition in a full grammar and you've tripled its length without adding a single fact. So we keep the shorthand. Just remember it's shorthand, because when you translate the rule into code, the `*` is what becomes your loop.

Seven rules can describe infinitely many valid programs. The trick is recursion. A `primary` can contain a whole `expression` again, so the grammar re-enters itself. Follow the loop once and you can see it close:

```
1 expression → equality → comparison → term → factor → unary → primary
2 primary    → "(" expression ")"
```

By the last line, you're back where you started, one level deeper. A fresh `expression` sits ready to expand however you like. Nothing stops you doing it again, so `(((((1)))))` is legal and so is a parenthesized expression nested a thousand deep. You never wrote a rule for "expression nested a thousand deep." You wrote a rule that mentions itself. The nesting came free.

That's also a warning about your own design. Any rule that can reach itself generates an infinite family of programs, so a rule that reaches itself *without consuming a token first* generates an infinite family of nothing and hangs your parser. Left recursion, as in `term → term "+" factor`, is that trap. It's why the textbook grammar loops with `*` instead.

Trace `2 + 3` by hand once, with pen and paper, before you write any code:

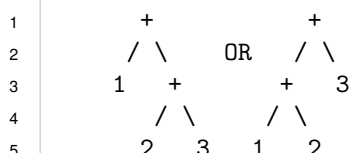
1. Start at `expression`.
2. Replace it with `equality`, the only choice.
3. `equality` becomes `comparison`, since there's no `==` or `!=` here.
4. `comparison` becomes `term`, since there's no comparison operator.
5. `term` becomes `factor ( ( "-" | "+" ) factor )*`.
6. The first `factor` descends through `unary` to `primary` and lands on `NUMBER(2)`.
7. There's a `+`, so the repetition runs once.
8. The second `factor` descends to `NUMBER(3)`.
9. You end with a `term` holding `2 + 3`.

Ambiguity, Precedence, and Associativity

A grammar can be ambiguous. An ambiguous grammar is useless to a parser. Here's the smallest example:

```
1 expression → expression "+" expression | NUMBER
```

For `1 + 2 + 3`, that rule allows two different trees:



The left tree means `1 + (2 + 3)`, the right means `(1 + 2) + 3`. Addition doesn't care, but swap the operator for subtraction or division and the two trees give different answers. Your parser has to pick one, so your grammar has to say which.

Look again at the textbook grammar and notice it's stratified from lowest precedence at the top to highest at the bottom. Equality binds loosest, then comparison, then addition and subtraction, then multiplication and division, then unary operators, and finally `primary`, where literals and explicitly parenthesized groups live. Each precedence level gets its own rule. Each rule delegates to the level below it. That's separation of concerns from CMSC 22, applied to syntax.

It's also what guarantees $2 + 3 * 4$ parses as $2 + (3 * 4)$. Don't take that on faith. Once you've seen the mechanism, you can extend it to any operator you invent, so here it is.

Walk the tokens $2 + 3 * 4$ through the two rules that matter:

```
1 term  → factor ( ( "-" | "+" ) factor ) *
2 factor → unary ( ( "/" | "*" ) unary ) *
```

Start in `term`. Before `term` may look at a single operator of its own, it has to call `factor` to get its left operand. So `factor` runs first, on the tokens starting at 2. It grabs 2, then checks whether the next token is `*` or `/`. It's `+`, which `factor` has no rule for, so `factor` stops and hands 2 back.

Now `term` is holding 2 and sees `+`. It consumes the operator and calls `factor` again for the right operand. This is the moment everything turns. `factor` starts at 3, grabs it, checks the next token, and this time it *is* `*`. So `factor` consumes the `*`, takes 4 as well, builds $(3 * 4)$, and only then hands that finished subtree back. `term` never saw the `*`. Control had returned by then. The work had already happened inside `factor`.

`term` combines the 2 it was holding and the subtree it was given into $(+ 2 (3 * 4))$. The multiplication is deeper in the tree. It was built first.

Say the mechanism plainly. It's the transferable part: **a lower-precedence rule can only ever receive finished higher-precedence subtrees**. It calls down for its operands, and whatever comes back has already swallowed everything that binds tighter than the caller. Precedence isn't checked anywhere. There's no table. No comparison of numbers. No special case. It's a consequence of call order.

Which tells you what to do when you add an operator to your own language. Decide what it binds tighter than. Then insert a rule between that rule and the one below it, delegating downward. Exponentiation binding tighter than `*` becomes a new rule sitting between `factor` and `unary`, with `factor` now calling it. No other edits. The precedence is correct by construction.

Associativity is the other half. It falls out of how you write the repetition. An operator is **left-associative** when a run of it groups from the left, so $1 - 2 - 3$ means $(1 - 2) - 3$. It's right-associative when the run groups from the right, giving $1 - (2 - 3)$. For subtraction those are 2 and negative 4, so this isn't a stylistic question:

```
1      -           -
2      / \        / \
3      - 3        1  -
4      / \        / \
5      1  2        2  3
6
7      LEFT      RIGHT
8      (1 - 2) - 3    1 - (2 - 3)
9      = -4          = 2
```

Almost every binary operator you'll write is left-associative. The loop form gives you that for free. Keep a variable holding the tree you've built so far. Each time round the loop, make that variable the *left* child of the new node:

```
1 expr = factor()
2 while next token is "-" or "+":
3     operator = advance()
4     right = factor()
5     expr = BinaryNode(expr, operator, right)    // old tree becomes the left child
```

After $1 - 2$ the variable holds $(- 1 2)$. The loop runs again for $- 3$ and wraps that whole subtree as the left operand, giving $(- (- 1 2) 3)$. The tree grows down the left side. That's what left-associative means.

Right associativity needs recursion. The operand on the right has to be the whole rest of the expression:

```

1 left = unary()
2 if next token is "^":
3     operator = advance()
4     right = power()           // calls itself, swallowing everything to the right
5     return BinaryNode(left, operator, right)

```

Exponentiation and assignment are the usual right-associative operators, so $a = b = c$ means $a = (b = c)$. If your language has one, write it recursively and say so in your grammar.

Recursive Descent

You'll implement a **recursive descent parser**. Both words in that name are doing work.

Descent is the direction. Parsing starts at the loosest-binding rule, *expression*, and calls downward toward the tightest, *primary*. That's where the actual tokens finally get consumed. It feels backwards the first time you see it, since the outermost function handles the operator that binds least and the deepest function handles the literals. The trace above is why it has to be that way. An outer rule can only assemble its node after its operands are finished, so it has to call down and wait. The rule you start from is the one whose node ends up at the *top* of the tree, which is also the last node built.

This is what "**top-down**" means when you see it in a compilers textbook. You begin from the grammar's starting rule and work toward the tokens, deciding what to build before you've seen everything you're building it from. The other family, bottom-up, starts from the tokens and assembles upward until it reaches the start rule. Bottom-up parsers handle more grammars and are what tools like yacc and bison generate, but nobody writes one by hand, and their error messages are famously bad. Top-down is the one you can debug with a print statement.

Recursive is the reason it terminates on nested input. A rule that mentions itself, directly or through *primary* looping back to *expression*, becomes a function that calls itself. The host language's call stack does all the bookkeeping about how deep you are. You never track nesting depth by hand.

Its main virtue is that your code ends up looking like your grammar. One rule, one function, in the same order, which makes it the friendliest parsing technique to learn and to debug. Don't mistake friendly for weak. GCC, several JavaScript engines, and the C# compiler all parse with recursive descent, because it's fast, it gives good error messages, and you can read it. If C++ can be parsed this way, your language can.

The *term* rule from above:

```

1 term → factor ( ( "-" | "+" ) factor )*

```

becomes a *term* function that calls *factor* for the left operand, then, while the current token is $+$ or $-$, saves the operator, calls *factor* again for the right operand, builds a binary node from the three pieces, and makes that node the new left operand. When the loop ends, it returns whatever it accumulated.

The Whole Month in One Example

Everything above arrived in pieces. Here it is running end to end on one input, from what your scanner hands over to what your printer prints. This is also the smallest useful thing you can test once the code exists.

Your Lab 1 scanner takes the source text $2 + 3 * 4$ and produces five tokens plus a terminator:

```
1  NUMBER(2)  PLUS  NUMBER(3)  STAR  NUMBER(4)  EOF
```

Your parser is handed that list and a position starting at 0. The calls go like this, indented by depth:

```
1  expression()
2    equality()          no "==" or "!=" ahead, so pass through
3      comparison()     no comparison operator, pass through
4        term()
5          factor()      LEFT OPERAND
6            unary()     no "!" or "-", pass through
7              primary() consumes NUMBER(2), returns Literal(2)
8                sees "+", so stop and return Literal(2)
9          sees "+", consume it
10         factor()      RIGHT OPERAND
11           unary() -> primary() consumes NUMBER(3), returns Literal(3)
12             sees "*", consume it
13           unary() -> primary() consumes NUMBER(4), returns Literal(4)
14             builds Binary(Literal(3), STAR, Literal(4))
15             sees EOF, stop and return that subtree
16         builds Binary(Literal(2), PLUS, Binary(3 * 4))
17         sees EOF, stop and return
```

Notice that `primary` is the only function that consumed a number, that `factor` consumed the `*` while `term` consumed the `+`, and that every function returned a finished node to its caller. The tree that comes back:

```
1      Binary(+)
2      /      \
3  Literal(2)  Binary(*)
4              /      \
5      Literal(3)  Literal(4)
```

Hand that to the printer you'll build later this month. It walks the tree writing the operator before its operands:

```
1  (+ 2.0 (* 3.0 4.0))
```

That string is what your test file will hold. Reading it back tells you the multiplication is nested inside the addition. That nesting was the whole point. If you get `(* (+ 2.0 3.0) 4.0)` instead, your `term` and `factor` are the wrong way round.

Learning Objectives

By the end of this activity, you should be able to:

- Write an unambiguous context-free grammar for your own language, with precedence and associativity encoded in its structure
- Read a grammar in both directions, generating a valid program from its rules and recognizing whether a given token sequence could have been generated
- Explain why an ambiguous grammar can't drive a parser, using an example from your own design

- Explain how stratified rules produce correct precedence without any precedence check in the code
- Translate each production rule mechanically into a recursive descent function
- Represent expressions as a tree of typed nodes in your host language
- Print a tree back out in an unambiguous textual form, and explain why that's the first debugging tool you should build
- Detect syntax errors, report them with position information, and exit with the code the run contract requires
- Keep an earlier activity's tests passing while adding a new pipeline stage above it

Task

Extend the repository from Lab 1 with a parser and an AST printer.

The tested path is `./run --parse <path-to-source-file>`, which scans a file, parses it, and prints the resulting tree in parenthesized form. The demonstrated path is the REPL, which should now print the parse of what you type. Your `--tokenize` flag stays exactly as it was, and `tests/lab1/` has to keep passing.

You'll also add a Grammar section to your `README.md` holding the CFG for your language as it now stands.

Required Features

Context-Free Grammar

Write the grammar for your language and add it to your `README.md` under a Grammar section. It has to be unambiguous, it has to encode your precedence and associativity choices in its structure, and it has to describe the language you implemented. Grammar and code drifting apart is the single most common way this activity goes wrong.

The textbook grammar is a reasonable skeleton, but a grammar copied without changes describes Lox. If your language uses `and` instead of `&&`, or has an exponentiation operator, or uses different comparison words, the grammar has to say so.

Parser

Implement a recursive descent parser that takes the token list from your scanner and produces a tree. It must handle:

- Literals: numbers, strings, and whatever boolean and nil-like values your language has
- Unary operators, including chained ones such as `!!false`
- Binary operators at every precedence level your language defines
- Explicit grouping with parentheses
- Correct associativity for every binary operator

Abstract Syntax Tree

You need node types for the constructs your parser produces. At minimum: a literal node carrying a value, a unary node carrying an operator and one operand, a binary node carrying an operator and two operands, and a grouping node carrying one inner expression.

The fields each one holds:

```

1 Literal { value }           // the number, string, or boolean itself
2 Unary   { operator, right } // operator is a Token, right is a node

```

```

3 Binary { left, operator, right } // both operands are nodes
4 Grouping { expression } // one inner node

```

The operands are nodes. That's what makes the structure a tree and lets `right` be an entire sub-expression. And `operator` is the whole **token**, since the token carries the line number and Lab 3 needs it to say where a runtime error happened. Storing only the symbol costs you nothing today and costs you a miserable afternoon next month.

How you represent these depends on your host language. Four answers cover almost every group. This manual's examples assume the first:

- A **class hierarchy**, where each node type is a subclass of a common `Expr` base. This is the Java-flavored answer and the one the textbook uses.
- A **tagged union**, one type that can hold any one of several alternatives, with a tag saying which alternative is currently inside. This is what a Rust `enum` with payloads or a C++ `std::variant` gives you, and it's the natural fit for a language with pattern matching.
- A **sealed interface**, an interface whose set of implementations is fixed and known to the compiler, so the compiler can check that you handled every case. Kotlin's `sealed` and C#'s newer pattern matching both do this.
- A **struct with a type tag**, an integer or enum field naming the kind, plus the fields for all kinds. The oldest answer and still reasonable in Go or C.

Pick the idiom your host language encourages, and be ready to explain the choice.

A word on the **visitor pattern**, since it gets mentioned constantly around this material and is rarely explained. The problem it solves is this. You have a fixed set of node types and a growing set of operations over them. This month you want to print a tree. Next month you'll want to evaluate one. After that you might want to optimize one or pretty-print it back to source. The obvious approach is to give each node class a `print` method, then add an `evaluate` method, then an `optimize` method, and now every new operation means editing every node class, and each class is a pile of unrelated logic that happens to share a data structure.

The visitor pattern inverts that. Each operation becomes one object, a "visitor," with one method per node type. `AstPrinter` has `visitLiteral`, `visitUnary`, `visitBinary`, and `visitGrouping`. Adding evaluation next month means writing a new visitor beside it, and touching no node class at all. The cost is ceremony. Each node needs an `accept` method that calls the right method on the visitor it's handed, which exists only so that the correct one gets chosen at runtime.

You don't have to use it. If your host language has pattern matching, a single `match` over node types does the same job with less machinery and reads better. Rust, Kotlin, and Julia groups should reach for that first. Understand the pattern anyway. Lab 3 extends whatever you build here. A visitor you copied without understanding is a visitor you can't extend under time pressure.

AST Printer

Build a printer that walks the tree and returns a string. Use prefix parenthesized form, where the operator comes first and its operands follow:

```

1 (+ 1.0 (* 2.0 3.0))

```

That form beats something that looks like source code because of one property. In prefix parenthesized form, the structure is fully explicit. Every operator sits inside a parenthesis with exactly its own operands, so you can read the grouping straight off the page without knowing a single thing about your language's precedence rules. Print `1 + 2 * 3` back in ordinary infix form and you've learned nothing. Reading it correctly requires the very rules you're trying to debug. Print it as `(+ 1.0 (* 2.0 3.0))` and the answer is right there in the brackets. It's the same reason Lisp looks the way it does.

This is your primary debugging tool for the rest of the semester and the thing your tests compare against. That's why it comes before evaluation. A tree you can't print is a tree you can't check.

Two details make it testable. Decide how a parenthesized group appears, since the tree no longer contains parentheses. The textbook prints `(group ...)`. And decide how numbers print, because `1` and `1.0` are different strings even when they're the same value. Write both decisions in your `README.md` and don't change them casually afterward.

Multiple Expressions in One File

Your language has no statements yet, so a test file is a sequence of expressions with nothing to separate them. Decide how a file gets split. One expression per line is the simplest rule and it's what the examples in this manual assume. Requiring a terminator such as `;` is also fine. It will save you an edit in Lab 4. Either way, `--parse` prints one line of output per expression parsed, and the rule goes in your `README.md`.

Error Reporting

Code has syntax errors. A parser that crashes on the first one is useless. When your parser meets a token it can't use, it should report what it expected and where, avoid taking down the process, and keep going so it can report more than one problem per file.

Report on `stderr`, print nothing to `stdout` for a file you rejected, and exit **65**. Something like this, from the textbook:

```
1 [line 1] Error at end: Expect ')' after expression.
2 [line 1] Error at 'this': Expect expression.
```

Continuing after an error is called **error recovery**. The usual technique is **synchronization**. On hitting an error, discard tokens until you reach one that plausibly begins a fresh construct, then resume. Without a statement boundary to synchronize on, you're limited to whatever separator you chose above. That's fine for now. Sophisticated recovery comes in Lab 4, once you have statements. What matters this month is that one bad expression doesn't lose you the rest of the file.

The REPL

The REPL now prints the parse of each line. A syntax error prints its message and returns the prompt. As in Lab 1, the harness never drives this path, so it's graded by demonstration.

The Run Contract

Nothing about the contract from Lab 0 changes. `./build.sh` builds once, program output goes to `stdout`, diagnostics go to `stderr`, and the exit codes stay 0 for accepted, 65 for rejected before execution, and 70 for died partway through (still unused until Lab 3).

What's new is a second flag alongside the first:

```
1 ./run --tokenize tests/lab1/keywords.mylang # unchanged from Lab 1
2 ./run --parse   tests/lab2/precedence.mylang # new this activity
```

Both stay for the rest of the semester. This is the pattern the whole laboratory sequence uses. Every pipeline stage keeps its own entry point, so every stage stays testable after the next one lands on top of it. When your Lab 4 parser gains statements and something subtle breaks in expression parsing, `tests/lab2/` tells you within a minute of pushing.

Name the flag whatever you like, record it in your `README.md`, and remember the harness passes exactly one flag per folder.

Writing Tests

Create `tests/lab2/` with its own manifest. Different folders can use different manifests, so this one sets a different flag from the Lab 1 folder:

```
1 {
2   "ext": ".mylang",
3   "flag": "--parse",
4   "mode": "sidecar"
5 }
```

Sidecar again, and for the same reason as last month. The strings you're checking are vocabulary your group invented, so there's no external answer key. The check is a regression check against expectations your group read and signed off on. Inline `// expect:` annotations arrive in Lab 3. That's when the values under test stop being your syntax and start being computed results.

A test is a source file plus its expected output, nothing more. `tests/lab2/precedence.mylang`:

```
1 2 + 3 * 4
2 (2 + 3) * 4
3 1 - 2 - 3
4 -(4 + 5)
```

Run `./run --parse` on it, read the four lines that come back, and satisfy yourself that each tree is the one you meant. Then commit that output verbatim as `tests/lab2/precedence.expected`:

```
1 (+ 2.0 (* 3.0 4.0))
2 (* (group (+ 2.0 3.0)) 4.0)
3 (- (- 1.0 2.0) 3.0)
4 (- (group (+ 4.0 5.0)))
```

Read those four lines closely. They're the whole activity in miniature. The first proves multiplication binds tighter. The second proves your grouping node exists and does something. The third proves subtraction is left-associative, where `(- (- 1 2) 3)` is right and `(- 1 (- 2 3))` is wrong. It takes a second to see which one you're looking at. The fourth proves a unary operator can take a group.

Rejections work the same way as in Lab 1. `tests/lab2/unbalanced.mylang`:

```
1 (1 + 2 + 3
```

Its `.expected` is empty, since nothing legitimate reached stdout, and `tests/lab2/unbalanced.exit` contains:

```
1 65
```

Remember what sidecar mode does and doesn't check: stdout and the exit code, never stderr. That test proves your parser rejected the input. Message quality is graded by a human at defense.

Run both folders locally before you push:

```

1  curl -sSL https://raw.githubusercontent.com/WhiteLicorice/cmsc-124-harness/v1.1/run_tests.py
    ↪ -o run_tests.py
2  ./build.sh
3  python run_tests.py tests/lab1
4  python run_tests.py tests/lab2

```

Use python3 on Linux and macOS, and Git Bash on Windows. Each test file gets 15 seconds before it's killed as a timeout.

If you change how your printer formats numbers or groups, every `.expected` file in `tests/lab2/` changes with it. Regenerating them is a two-minute job and reviewing the regenerated output honestly is not, so decide your format early and leave it alone. Committing regenerated expectations you didn't read is how a group ends up defending output they've never looked at.

Continuous Integration

Add the new folder to your workflow, keeping the old lines:

```

1  - run: python3 run_tests.py tests/lab0
2  - run: python3 run_tests.py tests/lab1
3  - run: python3 run_tests.py tests/lab2

```

All three have to be green. A red `tests/lab1` on a commit whose message says “feat: add parser” is the signal this setup exists to produce. You refactored the scanner while wiring up the parser and changed its output without meaning to. Fix the scanner, or, if the change was on purpose, regenerate the Lab 1 expectations, read them, and say so in your commit message.

Book your defense only after the Actions tab shows a green check on the commit you intend to defend.

Implementation Notes

The Parser's Own State

A recursive descent parser carries two pieces of state, the token list and an index into it. Nothing else. Where the nesting has got to and what has been built so far both live on the host language's call stack. That's what the recursion buys you.

Everything on top of those two is five small helpers, the token-level versions of the character helpers you wrote for the scanner. Almost every line of your parser is a call to one of them, so write all five before you write a single grammar rule:

```

1  function peek():                // current token, not consumed
2      return tokens[current]
3
4  function advance():             // consume and return it
5      token = tokens[current]
6      if not atEnd() : current = current + 1
7      return token
8
9  function check(type):           // is the current token this type?
10     if atEnd() : return false

```

```

11     return peek().type == type
12
13 function match(types...):           // consume only if it matches one of them
14     for type in types:
15         if check(type):
16             advance()
17             return true
18     return false
19
20 function consume(type, message): // demand this token, or fail loudly
21     if check(type) : return advance()
22     throw error(peek(), message)

```

`match` is the workhorse. It's what a `|` in the grammar becomes. It's also what drives every loop in your binary operator rules, since `while match(MINUS, PLUS)` reads almost exactly like `( ( "-" | "+" ) ... )*`. Note that it consumes the token only when it matches. That's what lets you ask "is it one of these?" without losing your place when the answer is no.

`consume` is the only one that can fail. It's where every syntax error in your parser originates. That message argument is the entire quality of your error reporting, so write it as the thing the programmer needs to hear. `consume(RIGHT_PAREN, "Expect ')' after expression.")` tells them what to fix. `consume(RIGHT_PAREN, "Parse error")` tells them nothing.

Turning Rules into Functions

With those helpers in hand, the translation from grammar to code is mechanical enough to be worth memorizing. These four patterns cover almost everything.

A choice between alternatives, as in `unary → ( "!" | "-" ) unary | primary`:

```

1 function unary():
2     if match(BANG, MINUS):
3         operator = previous()
4         operand = unary()           // recursive, so !!x works
5         return UnaryNode(operator, operand)
6     return primary()

```

Zero or more repetitions, as in `term → factor ( ( "-" | "+" ) factor )*`:

```

1 function term():
2     expr = factor()
3     while match(MINUS, PLUS):
4         operator = previous()
5         right = factor()
6         expr = BinaryNode(expr, operator, right) // left-associative fold
7     return expr

```

One or more repetitions, as in `arguments → expression ( "," expression )+`:

```

1 function arguments():
2     args = []
3     args.add(expression())           // the required first one
4     while match(COMMA):
5         args.add(expression())
6     return args

```

An optional element, as in function $\rightarrow$ "fun" IDENTIFIER "(" parameters? ")" block:

```

1 function parseFunction():
2     consume(FUN, "Expect 'fun'.")
3     name = consume(IDENTIFIER, "Expect function name.")
4     consume(LEFT_PAREN, "Expect '(' after function name.")
5     params = check(RIGHT_PAREN) ? empty list : parameters()
6     consume(RIGHT_PAREN, "Expect ')' after parameters.")
7     body = block()
8     return FunctionNode(name, params, body)

```

`previous()` is a sixth trivial helper returning the token `match` just consumed. You need it because `match` reports only whether it matched.

Underneath the individual patterns sits one more pattern. `*` becomes a while loop, `+` becomes one call then a while loop, `?` becomes an if, and `|` becomes a `match` over the alternatives. You won't need the last two until Lab 5, but they're here so you can see that the technique doesn't change as the grammar grows.

Errors as Control Flow

When `consume` fails, you're several function calls deep in the grammar. Every frame above you is mid-rule. Unwinding cleanly by returning error values through all of them is possible but tedious. Most implementations, including `jlox`, throw an exception and catch it at the top of the current construct, then synchronize and continue. Result types work too if your host language prefers them. Rust's `?` operator makes them pleasant. Either way, be deliberate. A parser that reports one error and stops passes fewer of your own tests than one that recovers.

Keeping the Tree Honest

The tree is the interface between this activity and the next one, so design it properly. Nodes should carry the token they came from, because Lab 3 needs the line number to report a runtime error and reconstructing it later is miserable. Grouping nodes exist for the printer's benefit and evaluate to their inner expression in Lab 3. That's a hint about how little they need to carry.

Common Pitfalls

- A grammar in the `README.md` that doesn't match the parser in the source. Update both in the same commit.
- Ambiguity you didn't notice. If two of your rules can both begin with the same token and the parser has to guess, you have a problem the grammar should have solved.
- Right-associative parsing where you meant left. Check `1 - 2 - 3` and read the printed tree carefully, because both readings look plausible at a glance.
- A primary rule that doesn't handle every literal your scanner can produce, which usually surfaces as "Expect expression" on perfectly valid input.
- A unary rule that isn't recursive, so `--10` or `!!true` fails.
- Forgetting to consume the closing parenthesis in a group, which makes the next expression start one token late.
- Infinite loops during error recovery. If synchronization doesn't consume at least one token, your parser will report the same error until the harness kills it at 15 seconds.
- Printing errors to `stdout`. They're diagnostics and `stdout` is compared byte for byte.
- Number formatting drift. If `--parse` prints `1` today and `1.0` tomorrow, every expectation in the folder is wrong.
- Breaking `tests/lab1` while refactoring the scanner and not noticing because you only ran the new folder locally. Run both.

Testing Strategy

Your `tests/lab2/` folder should include at least one file for each of these:

- Every literal type your language has, each printing as its own single line
- Precedence between at least three levels, such as equality against comparison against multiplication
- Left associativity, using an operator where the two readings differ, so subtraction or division
- Grouping that changes the tree, and a redundant group that doesn't
- Chained unary operators
- A long mixed expression combining four or more precedence levels, which is where a subtly wrong grammar finally shows itself
- Whatever your language has that Lox doesn't

Then the rejections, each with an `.exit` file containing 65:

- An unclosed parenthesis
- A binary operator missing its right operand, such as `1 +`
- A token that can't begin an expression
- An empty file or a file holding only a comment, if your design says that's an error. If your design says it's fine, test it as a success case with empty expected output instead

That last item is a design question. Nystrom's jlox reports "Expect expression" for input that's only a comment. It demands one expression. If your language treats an empty program as valid, then the test asserts exit 0 and an empty `.expected`. Either answer is defensible.

Hand-verify before you automate. For each precedence and associativity test, draw the tree on paper first, then compare it with what your printer produced. The tests protect the behavior. Only you can decide the behavior is right.

Deliverables

Your group's GitHub repository, presented during appointments, with a clean incremental commit history showing who wrote what, must contain:

1. Your parser and AST node definitions, plus the AST printer, documented where the code doesn't explain itself
2. A scanner that still works, with `--tokenize` intact
3. `build.sh` and `run` at the root, executable, supporting both flags and the no-argument REPL
4. `tests/lab2/` with `manifest.json`, test sources, and their `.expected` and `.exit` files
5. `.github/workflows/test.yml` running `tests/lab0`, `tests/lab1`, and `tests/lab2`, green on the commit you defend
6. A `README.md` following the specification template, with its Grammar section holding your unambiguous CFG, plus your parse output format, your file-splitting rule, and your parse flag

Then, each member submits, individually after the laboratory defense through email: a short `reflection.txt` about what you built and what fought back, plus a short `peer.txt` with your honest assessment of how your groupmates, including yourself, worked during the activity. Adhere to the following subject line: [CMSC 124 Lab] Lab 2: LastName, Initials, for example: [CMSC 124 Lab] Lab 2: Sanchez, SM. Include a link to your group's GitHub repository in the email. If even one member of a group fails to submit their individual deliverables, no final grade for the activity may be released for all members of the group.

Academic Honesty

Using large language models to generate wholesale vibe-coded submissions is cheating, subject to failure in the course and harsh disciplinary action. Recursive descent is a technique you can hold in your head, which makes this activity’s defense unusually easy to fail if you didn’t write the code. I’ll hand you an expression and ask which function runs first.

Important Dates

Progress reports and the laboratory defense may be booked only during the dates and hours defined in the syllabus. Book ahead through the booking page on the course site.

Activity	Monday	Tuesday
Week 1 Progress Report	Sep 21	Sep 22
Week 2 Progress Report	Sep 28	Sep 29
Week 3 Progress Report	Oct 5	Oct 6
Week 4 Laboratory Defense	Oct 12	Oct 13

Appointments need my verification before they count. If a date falls on a holiday or a suspension of classes, the syllabus allows a recorded progress report or defense for that seven-day period.

Note that this activity’s first week overlaps Lab 1’s defense window, which is by design and is why the syllabus caps you at two active activities. You may book one appointment per active activity each week. Groups that finish Lab 1 on schedule spend that overlapping week writing their grammar on paper. It’s the cheapest week of this activity. Skipping it costs you the most later.

For progress reports, Week 1 wants the grammar drafted and defensible, with a parser that handles literals and one binary precedence level. Week 2 wants every precedence level and the printer working, with tests committed alongside. Week 3 wants error recovery, green CI across all three folders, and your documented quirks.

Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Implementation Correctness (25%)	Every required feature of the activity works, including edge cases; the run contract is honored exactly, with program output on stdout, diagnostics on stderr, and the right exit code for each class of failure.	Core features correct with minor gaps in fringe cases; streams and exit codes right.	Core features work, but several requirements are missing or wrong; stream separation or exit codes are inconsistent.	Fails the activity's core requirement, ignores the run contract, or crashes with host-language errors on malformed input.
Testing and CI (20%)	Committed tests cover every required feature and its failure cases; expectations were reasoned out; every lab folder is green in CI on the defended commit.	Good coverage of the main features with at least one failure case; CI green.	Only happy paths tested, or an earlier lab folder left failing.	Almost no tests, no manifest, or no working workflow.
Code Engineering (10%)	Clean separation between pipeline stages; shared logic written once; constructs idiomatic for the host language; comments explain why.	Mostly clean structure with minor duplication or naming problems.	Monolithic functions; repeated logic; one stage leaking into another.	Unreadable, or the code contradicts the documented language design.
Collaboration (20%)	Professional Git usage: atomic, semantic commits from every member spread across the accomplishment period; tests and features arrive together; evidence of review or pair programming.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history, or history was rewritten or crammed.
Technical Defense (25%)	Every member traces their own code over fresh input, handles what-if questions, and defends the language design decisions behind the implementation.	Clear explanation of the implementation; both members participate meaningfully; logic delivery is sound.	Unclear explanations; uneven participation; struggles with code-tracing questions.	Unprepared; cannot explain the pipeline or defend design decisions.